

CSCE 411: Design and Analysis of Algorithms

Lecture 1: Intro, Asymptotic Runtimes, Divide and Conquer

Date: August 25, 2026

Nate Veldt, updated by Samson Zhou

Course Logistics

- Read section 2.3, chapter 4, and chapter 30 for first week of classes.
- Read (or skim) chapters 1-3 to ensure familiarity with prerequisites
- HW 1, intro video, and syllabus quiz are due Fri, Sep 4.

1 Computational Problems and Algorithms

Definition 1. A computational problem is a general task defined by a specific type of input and an explanation for a desired output

A specific case of the problem is called an instance

Example 1. Sorting

Input: A sequence of n numbers: $a_1, a_2, \dots, a_n$

Output: A permutation σ of the input sequence so that

$$a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}$$

An instance of this problem is the sequence 1, 0, -2, 3, 12
-2, 0, 1, 3, 12

Example 2. Min element

Input: An array of n numbers: $[a_1, a_2, \dots, a_n]$

Output: The smallest element in the array and its index.

instance [10, -1, 0, 12, 13]

Definition 2. An **algorithm** is a computational procedure that

- takes an input for a problem
- applies a concrete set of steps to produce a output

An **algorithm** is said to be **correct** if it always produces a valid output for each input

2 Asymptotic Runtime Analysis (Chapter 3)

2.1 Rules for runtime analysis

- n denotes the size of the input
- Each basic operation takes constant time
- We focus on the worst-case runtime

$\times, +, -, \div, \geq, \leq$
etc

- We only care about the order of the runtime

(asymptotic method)

2.2 An initial example

Question 1. Given an $n \times n$ matrix A , what is the runtime of summing the upper triangular portion using the following algorithm? (same answers).

```
sum = 0
for i = 1 to n do
  for j = i to n do
    sum = sum + aij
  end for
end for
Return sum
```

A $O(1)$

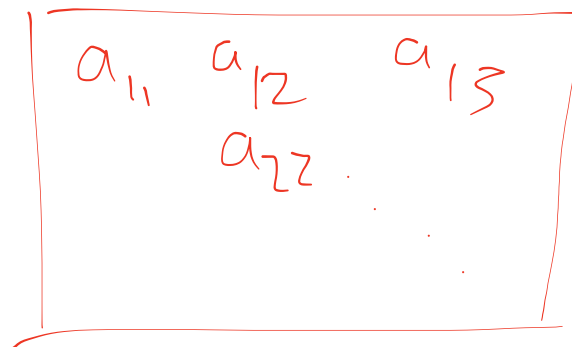
B $O(n)$

C $O(n^2)$

D It depends

E Other

F Don't know, I need a reminder for how this works.



$$n + (n-1) + \dots + 1$$

$$\frac{n(n+1)}{2}$$

2.3 Formal Definitions

Let n be input size, and let f and g be functions over $\mathbb{N}$.

Definition 3. Big O notation.

A function $g(n) = O(f(n))$ (we say, “ g is big- O of $f(n)$ ”) means:

there exists $c > 0$ and $n_0 \in \mathbb{N}$
Such that for every $n \geq n_0$,
 $g(n) \leq c \cdot f(n)$

Definition 4. Big Ω notation.

$g(n) = \Omega(f(n))$ means:

there exists $c > 0$ and $n_0 \in \mathbb{N}$
Such that for every $n \geq 0$, $g(n) \geq c \cdot f(n)$

Definition 5. Θ notation.

$g(n) = \Theta(f(n))$ means:

$g(n) = O(f(n))$ and
 $g(n) = \Omega(f(n))$

3 How to Present an Algorithm

Presenting and analyzing can be broken up into four steps.

1. **Explain:** the approach in basic English
2. **Pseudocode:** for formally presenting the algorithmic steps
3. Prove: the **correctness**
4. Analyze: the **runtime complexity**

As a rule it's a good idea to go through all steps when presenting an algorithm. Sometimes we will focus more on just a subset of these (e.g., you may be asked to prove a runtime complexity of an algorithm on a homework but not a correctness proof).

We will go through all four steps when we present the *merge sort* algorithm.

4 The Divide and Conquer Paradigm

The divide and conquer paradigm has three components:

- **Divide:** Split problem into subproblems
- **Conquer:** Solve subproblems individually
- **Combine:** combine subproblem solutions to overall solution

Example: Mergesort (Textbook, Chapter 2.3, 4) Given n numbers to sort, apply the following steps:

- Divide the sequence of length n into
- Recursively
- Combine

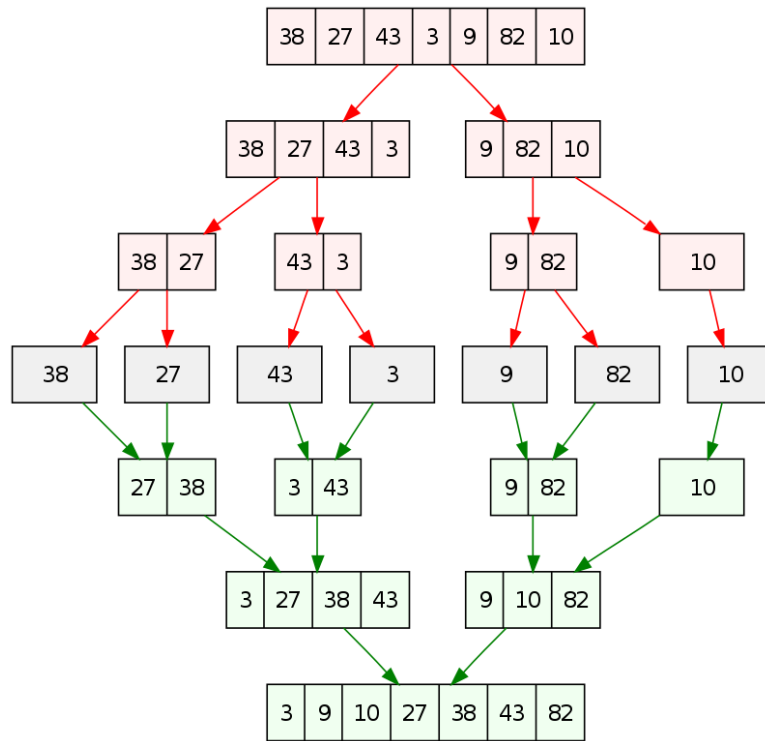


Image courtesy of Wikipedia: https://en.wikipedia.org/wiki/Merge_sort.

MERGESORT(A)

$n = \text{length}(A)$

if $n == 1$ **then**

else

$m = \lfloor n/2 \rfloor$

return A

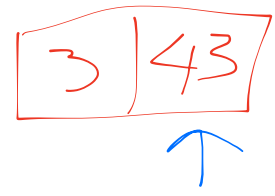
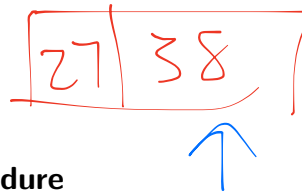
$L = A[1, m], \quad R = [m+1, n]$

$L' = \text{MergeSort}(L), \quad R' = \text{MergeSort}(R)$

Combine (L', R')

end if

Merge



4.1 Analyzing The Merge Procedure

Correctness: To merge two sorted subarrays into a master array

- Maintain a pointer to the end (max value) of L and R
- At each step, Compare the numbers from the subarrays
- Since subarrays are sorted, one of these numbers is guaranteed to be the largest among un-merged numbers
- so place in last open slot
- At each step, we guarantee that: largest remaining number is sorted
- So continuing until both subarrays are empty, gives sorted array

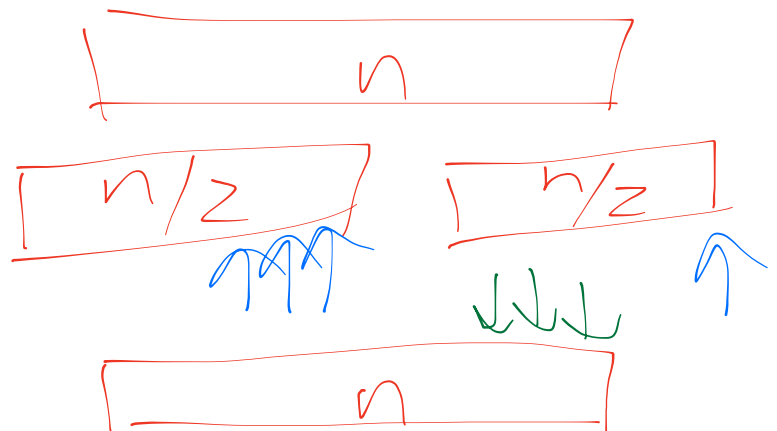
5 Continued Analysis of Merge Sort

Merge Sort. Given n numbers to sort

- Divide the sequence of length n into arrays of length $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$
- Recursively sort the two halves
- (Merge Procedure) Combine the two halves by sorting them. 

Question 2. What is the runtime of the *merge procedure* in Merge Sort?

- A** $\Theta(1)$
- B** $\Theta(n \lg n)$ 23%
- C** $\Theta(n)$ 77%
- D** $\Theta(n^2)$



$$2n = O(n)$$

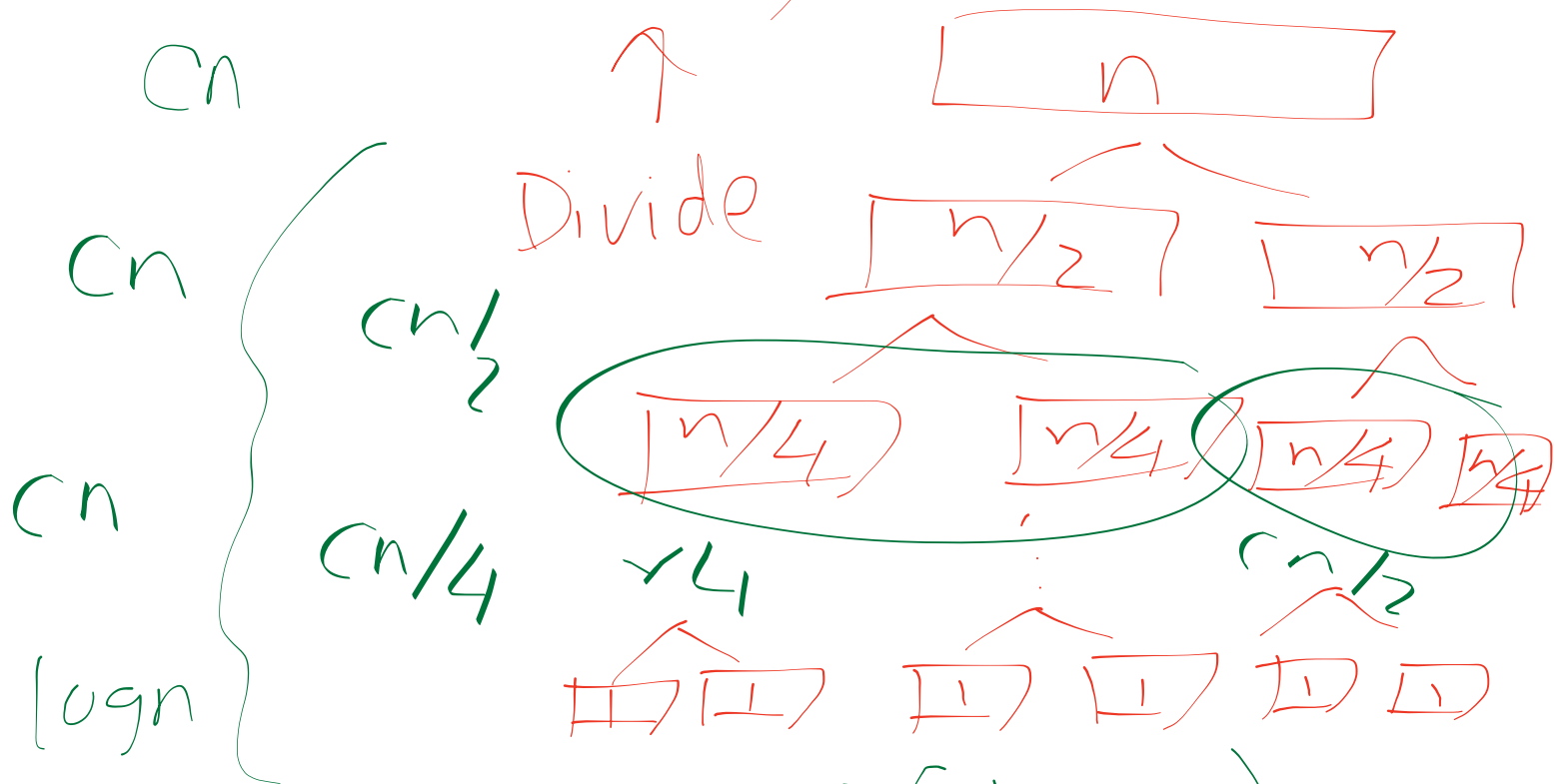
6 Recurrence Analysis for Divide and Conquer

Runtimes for divide and conquer algorithms can be described in terms of a recurrence relation, which lists runtime in terms of runtimes of smaller problems

Let $T(n)$ denote the runtime for a problem of size n .

Example: merge-sort Assume for this analysis that $n = 2^p$ where $p \in \mathbb{N}$.

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$



$$cn \log n = O(n \log n)$$

7 Three methods for solving recurrences

Given a recurrence relation, there are three approaches to finding the overall runtime.

- Recursion tree: as above (draw recursion tree)
- Substitution method: guess and compute and prove it runtime)
- Master theorem:

Someone else did it
for you

8 The Master Theorem for Recurrence Relations

Theorem 8.1. Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the relation:

of subproblems $T(n) = aT(n/b) + f(n)$ size of subproblems combine

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then

$$T(n) = \Theta(n^{\log_b a})$$

2. If $f(n) = \Theta(n^{\log_b a})$, then

$$T(n) = \Theta(n^{\log_b a} \log n)$$

3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then

$$T(n) = \Theta(f(n))$$

8.1 Example: Merge-Sort

Recall that Merge-Sort satisfies the following recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases} \quad (1)$$

We can apply the master theorem with:

$$a=2, b=2 \quad n^{\log_b a} = n \Rightarrow T(n) = \Theta(n \log n)$$

$$f(n) = \Theta(n)$$

8.2 What to know about the master method?

Proof idea:

Follows by applying the recursion tree method on a general recurrence relationship and doing casework

What's more important:

you know how apply it

8.3 Examples

Apply the master theorem to the following recurrences:

$$T(n) = 9T(n/3) + n \quad (2)$$

$$T(n) = 3T(n/4) + n \log n \quad (3)$$

$$T(n) = 7T(n/2) + \Theta(n^2) \quad (4)$$

$$1) \ a=9, b=3 \quad \log_b a = \log_3 9 = 2$$
$$n^{\log_b a} = n^2$$

$$f(n) = n \quad \text{1st case}$$

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$$

$$2) \ a=3, b=4 \quad \log_b a = \log_4 3 < 1$$
$$n^{\log_b a} < n \log n \quad \text{3rd case}$$

$$f(n) = n \log n$$

$$a f\left(\frac{n}{b}\right) = 3\left(\frac{n}{4}\right) \log \frac{n}{4} < n \log n$$

$$T(n) = \Theta(n \log n)$$

$$3) \ a=7, b=2, \log_b a = \log_2 7 \in (2, 3)$$
$$f(n) = n^2 \quad \text{1st case} \quad T(n) \sim \Theta(n^{2.37})$$